Exercise4_1

HDL Code

```
primitive demuxF0(F,Y,S2,S1,S0);

        input Y,S2,S1,S0;
        output F;

        table
        //      Y       S[2]    S[1]    S[0]    :       F
                0       b       b       b       :       0;
                1       0       0       0       :       1;
                1       0       0       1       :       0;
                1       0       1       0       :       0;
                1       0       1       1       :       0;
                1       1       0       0       :       0;
                1       1       0       1       :       0;
                1       1       1       0       :       0;
                1       1       1       1       :       0;

        endtable
endprimitive

primitive demuxF1(F,Y,S2,S1,S0);

        input Y,S2,S1,S0;
        output F;

        table
        //      Y       S[2]    S[1]    S[0]    :       F
                0       b       b       b       :       0;
                1       0       0       0       :       0;
                1       0       0       1       :       1;
                1       0       1       0       :       0;
                1       0       1       1       :       0;
                1       1       0       0       :       0;
                1       1       0       1       :       0;
                1       1       1       0       :       0;
                1       1       1       1       :       0;

        endtable
endprimitive
```

```
primitive demuxF2(F,Y,S2,S1,S0);

        input Y,S2,S1,S0;
        output F;

        table
        //      Y       S[2]    S[1]    S[0]    :       F
                0       b       b       b       :       0;
                1       0       0       0       :       0;
                1       0       0       1       :       0;
                1       0       1       0       :       1;
                1       0       1       1       :       0;
                1       1       0       0       :       0;
                1       1       0       1       :       0;
                1       1       1       0       :       0;
                1       1       1       1       :       0;

        endtable
endprimitive

primitive demuxF3(F,Y,S2,S1,S0);

        input Y,S2,S1,S0;
        output F;

        table
        //      Y       S[2]    S[1]    S[0]    :       F
                0       b       b       b       :       0;
                1       0       0       0       :       0;
                1       0       0       1       :       0;
                1       0       1       0       :       0;
                1       0       1       1       :       1;
                1       1       0       0       :       0;
                1       1       0       1       :       0;
                1       1       1       0       :       0;
                1       1       1       1       :       0;

        endtable
endprimitive
```

```
primitive demuxF4(F,Y,S2,S1,S0);

        input Y,S2,S1,S0;
        output F;

        table
        //      Y       S[2]    S[1]    S[0]    :       F
                0       b       b       b       :       0;
                1       0       0       0       :       0;
                1       0       0       1       :       0;
                1       0       1       0       :       0;
                1       0       1       1       :       0;
                1       1       0       0       :       1;
                1       1       0       1       :       0;
                1       1       1       0       :       0;
                1       1       1       1       :       0;

        endtable
endprimitive

primitive demuxF5(F,Y,S2,S1,S0);

        input Y,S2,S1,S0;
        output F;

        table
        //      Y       S[2]    S[1]    S[0]    :       F
                0       b       b       b       :       0;
                1       0       0       0       :       0;
                1       0       0       1       :       0;
                1       0       1       0       :       0;
                1       0       1       1       :       0;
                1       1       0       0       :       0;
                1       1       0       1       :       1;
                1       1       1       0       :       0;
                1       1       1       1       :       0;

        endtable
endprimitive
```

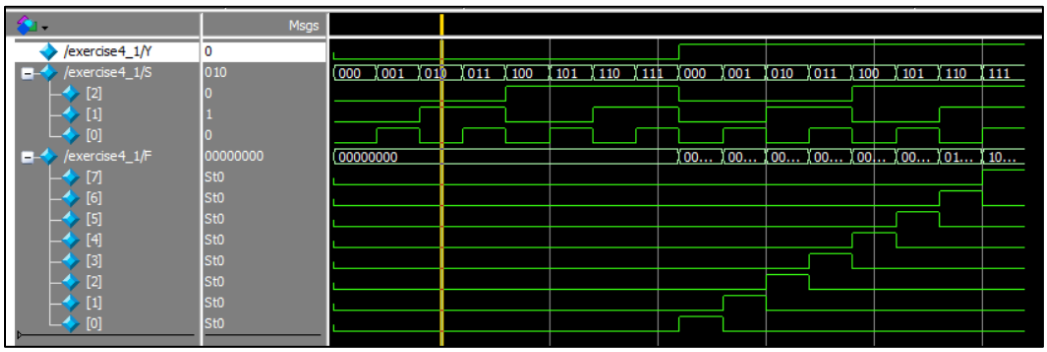
```
primitive demuxF6(F,Y,S2,S1,S0);

        input Y,S2,S1,S0;
        output F;

        table
        //      Y       S[2]    S[1]    S[0]    :       F
                0       b       b       b       :       0;
                1       0       0       0       :       0;
                1       0       0       1       :       0;
                1       0       1       0       :       0;
                1       0       1       1       :       0;
                1       1       0       0       :       0;
                1       1       0       1       :       0;
                1       1       1       0       :       1;
                1       1       1       1       :       0;

        endtable
endprimitive

primitive demuxF7(F,Y,S2,S1,S0);

        input Y,S2,S1,S0;
        output F;

        table
        //      Y       S[2]    S[1]    S[0]    :       F
                0       b       b       b       :       0;
                1       0       0       0       :       0;
                1       0       0       1       :       0;
                1       0       1       0       :       0;
                1       0       1       1       :       0;
                1       1       0       0       :       0;
                1       1       0       1       :       0;
                1       1       1       0       :       0;
                1       1       1       1       :       1;

        endtable
endprimitive
```

```
module exercise4_1;
        reg Y; reg[2:0] S;
        wire[7:0] F;

        demuxF0(F[0],Y,S[2],S[1],S[0]);
        demuxF1(F[1],Y,S[2],S[1],S[0]);
        demuxF2(F[2],Y,S[2],S[1],S[0]);
        demuxF3(F[3],Y,S[2],S[1],S[0]);
        demuxF4(F[4],Y,S[2],S[1],S[0]);
        demuxF5(F[5],Y,S[2],S[1],S[0]);
        demuxF6(F[6],Y,S[2],S[1],S[0]);
        demuxF7(F[7],Y,S[2],S[1],S[0]);

        initial begin
                Y=1'b0; S=3'b000;
                $display("Simulating 8X1 Demultiplexer...");
                $display("Y\tS\tF0\tF1\tF2\tF3\tF4\tF5\tF6\tF7");
                $monitor("%b\t%b\t%b\t%b\t%b\t%b\t%b\t%b\t%b\t%b\t%b",Y,S,F[0],F[1],F[2],F[3],F[4],F[5],F[6],F[7]);

                #2 Y=1'b0; S=3'b001;
                #2 Y=1'b0; S=3'b010;
                #2 Y=1'b0; S=3'b011;
                #2 Y=1'b0; S=3'b100;
                #2 Y=1'b0; S=3'b101;
                #2 Y=1'b0; S=3'b110;
                #2 Y=1'b0; S=3'b111;
                #2 Y=1'b1; S=3'b000;
                #2 Y=1'b1; S=3'b001;
                #2 Y=1'b1; S=3'b010;
                #2 Y=1'b1; S=3'b011;
                #2 Y=1'b1; S=3'b100;
                #2 Y=1'b1; S=3'b101;
                #2 Y=1'b1; S=3'b110;
                #2 Y=1'b1; S=3'b111;
                #2 $finish;
        end

endmodule
```

Transcript

```
VSIM 7> run
# GetModuleFileName: The specified module could not be found.
#
#
# Simulating 8X1 Demultiplexer...
# Y     S       F0      F1      F2      F3      F4      F5      F6      F7
# 0     000     0       0       0       0       0       0       0       0
# 0     001     0       0       0       0       0       0       0       0
# 0     010     0       0       0       0       0       0       0       0
# 0     011     0       0       0       0       0       0       0       0
# 0     100     0       0       0       0       0       0       0       0
# 0     101     0       0       0       0       0       0       0       0
# 0     110     0       0       0       0       0       0       0       0
# 0     111     0       0       0       0       0       0       0       0
# 1     000     1       0       0       0       0       0       0       0
# 1     001     0       1       0       0       0       0       0       0
# 1     010     0       0       1       0       0       0       0       0
# 1     011     0       0       0       1       0       0       0       0
# 1     100     0       0       0       0       1       0       0       0
# 1     101     0       0       0       0       0       1       0       0
# 1     110     0       0       0       0       0       0       1       0
# 1     111     0       0       0       0       0       0       0       1
# ** Note: $finish    : C:/intelFPGA_lite/20.1/User-Defined Primitives/Exercise4_1.v(194)
#    Time: 32 ps  Iteration: 0  Instance: /exercise4_1
```

Wave


Msgs
/exercise4_1/Y
/exercise4_1/S
[2]
[1]
[0]
/exercise4_1/F
[7]
[6]
[5]
[4]
[3]
[2]
[1]
[0]
0
010
0
1
0
00000000
St0
St0
St0
St0
St0
St0
St0
St0
000 001 010 011 100 101 110 111 000 001 010 011 100 101 110 111
00000000
00... 00... 00... 00... 00... 00... 01... 10...

Exercise4_2

HDL Code

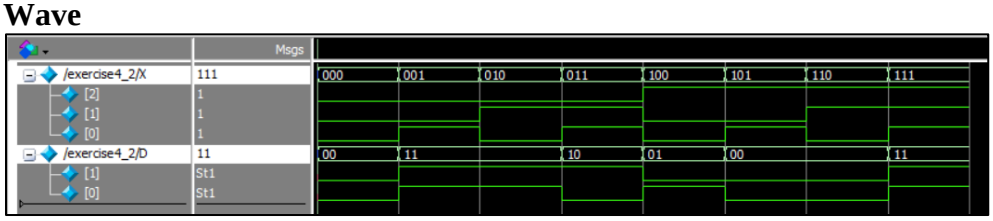
```
primitive Diff(D,A,B,Bin);

        input A,B,Bin;
        output D;

        table

        //      A       B       Bin     :       D
                0       0       0       :       0;
                0       0       1       :       1;
                0       1       0       :       1;
                0       1       1       :       0;
                1       0       0       :       1;
                1       0       1       :       0;
                1       1       0       :       0;
                1       1       1       :       1;

        endtable
endprimitive

primitive Bout(Bout,A,B,Bin);

        input A,B,Bin;
        output Bout;

        table


        //      A       B       Bin     :       Bout
                0       0       0       :       0;
                0       0       1       :       1;
                0       1       0       :       1;
                0       1       1       :       1;
                1       0       0       :       0;
                1       0       1       :       0;
                1       1       0       :       0;
                1       1       1       :       1;

        endtable
endprimitive
```

```
module exercise4_2;

        reg[2:0] X;
        wire[1:0] D;

        Diff(D[0],X[2],X[1],X[0]);
        Bout(D[1],X[2],X[1],X[0]);

        initial begin
                X=3'b000;
                $display("Simulating Binary Subtractor");
                $display("A\tB\tBin\tBout\tDiff");
                $monitor("%b\t%b\t%b\t%b\t%b",X[2],X[1],X[0],D[1],D[0]);

                #2 X=3'b001;
                #2 X=3'b010;
                #2 X=3'b011;
                #2 X=3'b100;
                #2 X=3'b101;
                #2 X=3'b110;
                #2 X=3'b111;
                #2 $finish;
        end

endmodule
```

Transcript

```
VSIM 14> run
# GetModuleFileName: The specified module could not be found.
#
#
# Simulating Binary Subtractor
# A     B       Bin     Bout    Diff
# 0     0       0       0       0
# 0     0       1       1       1
# 0     1       0       1       1
# 0     1       1       1       0
# 1     0       0       0       1
# 1     0       1       0       0
# 1     1       0       0       0
# 1     1       1       1       1
# ** Note: $finish    : C:/intelFPGA_lite/20.1/User-Defined Primitives/Exercise4_2.v(61)
#    Time: 16 ps  Iteration: 0  Instance: /exercise4_2
# 1
# Break in Module exercise4_2 at C:/intelFPGA_lite/20.1/User-Defined Primitives/Exercise4_2.v line 61
```

Wave



Drill4_1

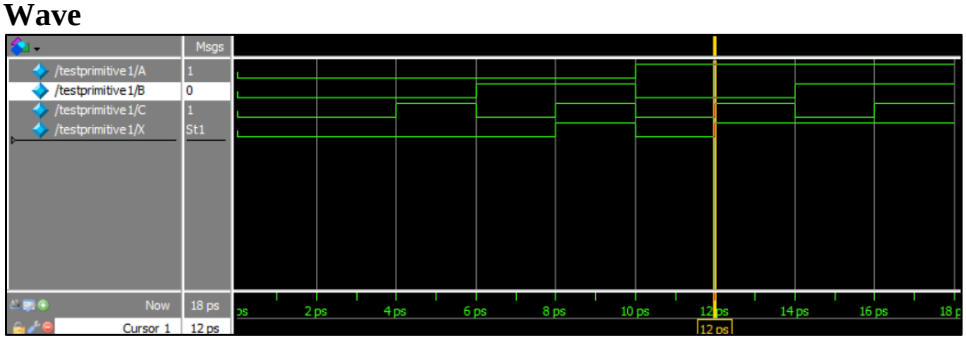
HDL Code

```
primitive Majority3 ( Z, A, B, C );

        input A, B, C ;
        output Z ;

        table
        // A B C Z
        0 0 ? : 0 ;
        0 ? 0 : 0 ;
        ? 0 0 : 0 ;
        1 1 ? : 1 ;
        1 ? 1 : 1 ;
        ? 1 1 : 1 ;
        endtable

endprimitive

module testprimitive1;

        reg A, B, C;
        wire X;
        Majority3 (X,A,B,C);

        initial begin
                A=1'b0;B=1'b0;C=1'b0;
                $display("A\tB\tC\tX");
                $monitor("%b\t%b\t%b\t%b",A,B,C,X);
                #2 A=1'b0; B=1'b0; C=1'b0;
                #2 A=1'b0; B=1'b0; C=1'b1;
                #2 A=1'b0; B=1'b1; C=1'b0;
                #2 A=1'b0; B=1'b1; C=1'b1;
                #2 A=1'b1; B=1'b0; C=1'b0;
                #2 A=1'b1; B=1'b0; C=1'b1;
                #2 A=1'b1; B=1'b1; C=1'b0;
                #2 A=1'b1; B=1'b1; C=1'b1;
                #2 $finish;
        end

endmodule
```

Transcript

```
VSIM 2> run -all
# A     B       C       X
# 0     0       0       0
# 0     0       1       0
# 0     1       0       0
# 0     1       1       1
# 1     0       0       0
# 1     0       1       1
# 1     1       0       1
# 1     1       1       1
# ** Note: $finish    : C:/intelFPGA_lite/20.1/Gate-Level Modeling/Drill4_1.v(31)
#    Time: 18 ps  Iteration: 0  Instance: /testprimitive1
```

Wave



Drill4_2

HDL Code

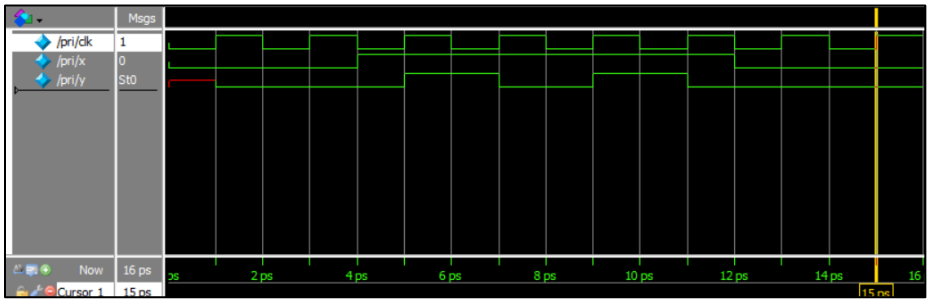
```
primitive T_FF(F, clk, T);

        input clk, T;
        output F;
        reg F;

        initial F=0;

        table
        // CLK T : F(STATE) : F(NEXT)
        (01) ? : x : 0 ;
        (01) 0 : 0 : 0 ;
        (01) 0 : 1 : 1 ;
        (01) 1 : 0 : 1 ;
        (01) 1 : 1 : 0 ;
        (10) ? : x : 0 ;
        (10) 0 : 0 : 0 ;
        (10) 0 : 1 : 1 ;
        (10) 1 : 0 : 0 ;
        (10) 1 : 1 : 1 ;
        endtable

endprimitive
```

```
module pri;

        reg x, clk;
        wire y;

        T_FF TF(y, clk, x);

        initial begin
                x=0; clk=0;
        end

        always #1 clk=!clk;

        initial begin
                x=0;
                #4 x=1;
                repeat(8)
                #8 x=~x;
        end

        initial begin
                $display("TIME\tClk\tT\tQ");
                $monitor("%0t\t%b\t%b\t%b",$time, clk, x, y);
                #16 $finish;
        end
endmodule
```

Transcript

```
VSIM 6> run
# GetModuleFileName: The specified module could not be found.
#
#
# TIME  Clk     T       Q
# 0     0       0       x
# 1     1       0       0
# 2     0       0       0
# 3     1       0       0
# 4     0       1       0
# 5     1       1       1
# 6     0       1       1
# 7     1       1       0
# 8     0       1       0
# 9     1       1       1
# 10    0       1       1
# 11    1       1       0
# 12    0       0       0
# 13    1       0       0
# 14    0       0       0
# 15    1       0       0
# ** Note: $finish    : C:/intelFPGA_lite/20.1/User-Defined Primitives/Drill4_2.v(48)
#    Time: 16 ps  Iteration: 0  Instance: /pri
```

Wave



Descriptive Explanation

Drill4_1

The code defines a Verilog User-Defined Primitive (UDP) named Majority3, which implements a majority gate logic for three inputs (A, B, C) and one output (Z). The table within the UDP specifies that Z will be 1 if at least two of the three inputs are 1, and 0 otherwise. The testprimitive1 module is used to simulate this UDP, where the inputs (A, B, C) are defined as reg types and the output X is connected to the UDP. Inside the initial block, various combinations of inputs are applied to test the Majority3 behavior, while $monitor continuously displays the input and output values during the simulation. The test runs for several time steps, applying different input patterns.

Drill4_2

The code defines a T Flip-Flop using a Verilog User-Defined Primitive (UDP) named T_FF, which has inputs clk (clock) and T, and an output F (flip-flop state). The flip-flop's behavior is described in the table, where the output toggles if T is 1 when there is a rising clock edge, otherwise it holds the previous state. In the pri module, the T_FF primitive is instantiated, and the clk signal is toggled every time unit using an always block, while the input x (T) alternates values periodically. The simulation displays the clock, input T, and output y at every time step using $monitor, and after 16-time units.

Exercise4_1

The Verilog code implements an 8x1 demultiplexer that routes the input Y to one of the eight outputs (F[0] to F[7]) based on the 3-bit select lines S[2:0]. Each output is controlled by a specific primitive (demuxF0 to demuxF7), which defines the behavior of that output depending on the combination of Y and S. For each combination of S[2:0], one output is set to the value of Y (if Y is 1), while the rest of the outputs remain 0. For instance, when S = 000, F[0] is assigned the value of Y, and when S = 001, F[1] takes the value of Y, and so on, ensuring that only one output is active at a time based on the select lines.

Exercise4_2

This Verilog code defines two primitives, Diff and Bout, which represent the difference and borrow output logic for a 1-bit binary subtractor. The Diff primitive computes the difference (D) based on inputs A, B, and Bin (borrow in), while Bout calculates the borrow out (Bout) under similar inputs. In the exercise4_2 module, a 3-bit input X is used to represent the values of A, B, and Bin, and the difference and borrow outputs are stored in a 2-bit wire D. The simulation iterates over all possible values of X, displaying the inputs and outputs at each step to demonstrate how the binary subtractor works. The initial block controls the simulation timing, updating the inputs every 2-time units and ending after all cases are tested.